

Rapport Projet de Programmation

Résolution du problème de plus court chemin sous contrainte de fatigue cumulative

Bastien LEVY-GUINOT et Niels SOUFFIR - Année scolaire 2025-2026 - 1A ENSAE Paris

Résumé

Nous étudions le problème d'optimisation de parcours au sein d'un réseau soumis à des contraintes de fatigue cumulative. La résolution de ce problème s'articule autour d'une montée en puissance algorithmique. L'objectif est de déterminer le chemin minimisant le temps de trajet total tout en respectant un seuil de fatigue. Nous modélisons d'abord ce problème naïvement par une extension de graphe permettant l'utilisation de l'algorithme de Dijkstra. Cette approche, bien que garante de l'optimalité, se confronte aux limites de l'exploration exhaustive et gagne rapidement en complexité sur des réseaux de grande taille. Constatant ces limites, nous explorons dans une seconde phase des optimisations algorithmiques telles que le passage à un graphe implicite, le pruning et l'algorithme A*. En particulier, nous concevons un algorithme qui résout le problème en $O(m \times L \times \log(n \times L))$, où, L est une constante décrite plus tard. Enfin, nous présentons les performances de nos implémentations et abordons l'extension du problème à des missions multiples.

Table des matières

I-Approche naïve

- a) Description du problème
- b) Reformulation du problème : le Graphe Étendu
- c) Première résolution : algorithme de Dijkstra

II- Améliorations algorithmiques

- a) Le graphe implicite : paradigme de l'évaluation paresseuse
- b) Pruning et dominance de pareto
- c) A* et heuristique

III- Résultats et extensions

- a) Benchmark des résultats et analyse des données
- b) Extension : Mission multiple
- c) Extension : Optimisation par colonies de fourmis (ACO)

I-Approche naïve

a) Soit un graphe orienté $G = (V, E)$, avec $\text{card}(E)=n$, $\text{card}(V)=m$. On cherche à minimiser le temps de trajet entre deux sommets s et t . Chaque arc $e = (u, v) \in E$ est pondéré par deux grandeurs : une longueur $l(e)$ et une fatigue $f(e)$. Le coût de franchissement d'une arête vaut longueur_arc $\times$ fatigue_accumulé. Ainsi, le temps total de parcours d'un chemin $(v_0, \dots, v_k)$ est définie comme : $l(v_0, v_1) + l(v_1, v_2) \times (1 + f(v_0, v_1)) + \dots + l(v_{k-1}, v_k) \times (1 + f(v_0, v_1) + f(v_{k-2}, v_{k-1}))$.

Cette dépendance directe entre le coût d'une arête et l'historique du parcours rend l'utilisation de l'algorithme de Dijkstra impossible sur le graphe initial.

b) Pour retrouver une propriété de stationnarité, il faut augmenter l'espace de recherche. Nous transformons la variable dynamique (la fatigue) en une coordonnée statique au sein d'un espace d'états augmenté, en construisant un graphe étendu dans lequel chaque sommet est un couple (sommet, fatigue), au moyen de *build_extended_graph* qui utilise une fonction de hachage. Dans cette nouvelle structure, on démultiplie une route reliant u à v dans le réseau d'origine en une série d'arêtes reliant chaque état (u, F) à l'état $(v, F + f_{uv})$, on connecte un noeud virtuelle à la cible pour pouvoir utiliser Dijkstra une seule fois, et on calcule un seuil maximale pour la fatigue ($F_{\max}$).

La complexité spatiale s'établit à $O((n+m) \times F_{\max})$: Chaque nœud d'origine est démultiplié en $F_{\max}$ sommets distincts et chaque arête est clonée $F_{\max}$ fois. La complexité temporelle est de $O(m \cdot F_{\max} + m \log m)$ car il y a le tri d'une liste ($m \log m$ pour *compute_F_max*) et une double itération: le parcours des m arêtes du réseau initial, au sein duquel s'exécute une boucle générant les transitions pour chaque unité de fatigue de 1 à $F_{\max}$. La modélisation par graphe étendu repose sur une duplication systématique des états pour chaque unité de fatigue, ce qui engendre une explosion des ressources nécessaires.

c) Grâce à la construction du graphe étendu, le problème de base est ramené à une recherche du pcc. Puisque les poids des arêtes sont positifs, l'algorithme de Dijkstra devient notre outil de résolution. Nous l'implémentons en utilisant une structure de tas : une approche gloutonne est utilisé. En extrayant systématiquement à chaque étape le sommet u qui minimise la distance provisoire, l'algorithme garantit l'optimalité du chemin trouvé sans nécessiter l'exploration exhaustive du graphe. Cette stratégie repose sur la propriété de stationnarité des distances : une fois qu'un sommet est extrait de la file de priorité, sa distance est considérée comme définitive.

Le tas binaire permet d'extraire le sommet le plus proche en temps logarithmique. Chaque sommet du graphe est extrait du tas une seule fois au maximum. Il y a au plus n sommets, et chaque extraction (*heappop*) coûte environ $(\log n)$. Pour chaque sommet extrait, on regarde tout ses arcs sortants (les arêtes m). Si on trouve un chemin plus court, on ajoute la nouvelle distance dans le tas (*heappush*), ce qui coûte aussi $\log n$. La complexité temporelle totale s'établit à $O(n) + O(m \log n) + O(n \log n) = O((n + m) \times \log n)$. La complexité spatiale est en $O(n+m)$. Ce coût mémoire vient de la nécessité de maintenir, d'une part, les tableaux de distances et de parents dont la taille est égale au nombre de sommets et, d'autre part, la file de priorité et la liste d'adjacence qui dépendent du nombre d'arêtes m .

II- Améliorations algorithmique

a) L'approche par graphe implicite marque une rupture avec la construction statique de l'espace d'états car on ne stocke aucune liste d'adjacence pour les états étendus. Elle repose sur une évaluation paresseuse : les connexions ne sont calculées qu'au moment où l'algorithme explore un sommet. Cette dynamique est portée par la méthode *_neighbors* : en utilisant l'instruction *yield*, elle transforme la fonction de voisinage en un générateur ; chaque voisin est produit, transmis à l'algorithme, puis immédiatement évacué de la mémoire une fois traité. Cette transition algorithmique transforme la complexité spatiale du problème. Dans l'approche explicite, le stockage des arêtes du graphe étendu imposait un coût en $O(m \times \text{Fatigue_max})$. Avec *build_extended_implicit_graph*, ce coût disparaît : l'occupation mémoire est ramenée à $O(n+m)$. Ce paradigme permet ainsi de simuler un graphe dont la taille exhaustive excéderait les capacités de la mémoire vive.

b) L'idée de l'élagage utilisé dans les fonctions *pruning* et *pruning_implicit* est de faire le tri pour ne pas explorer des chemins qui n'ont aucune chance d'être les meilleurs. On utilise la dominance de Pareto : un chemin est dominé s'il existe déjà une alternative à la fois plus courte et moins fatigante pour atteindre le même sommet. Pour chaque noeud du graph étendu, on maintient une frontière qui enregistre uniquement les meilleurs compromis distance/fatigue rencontrés jusque-là.

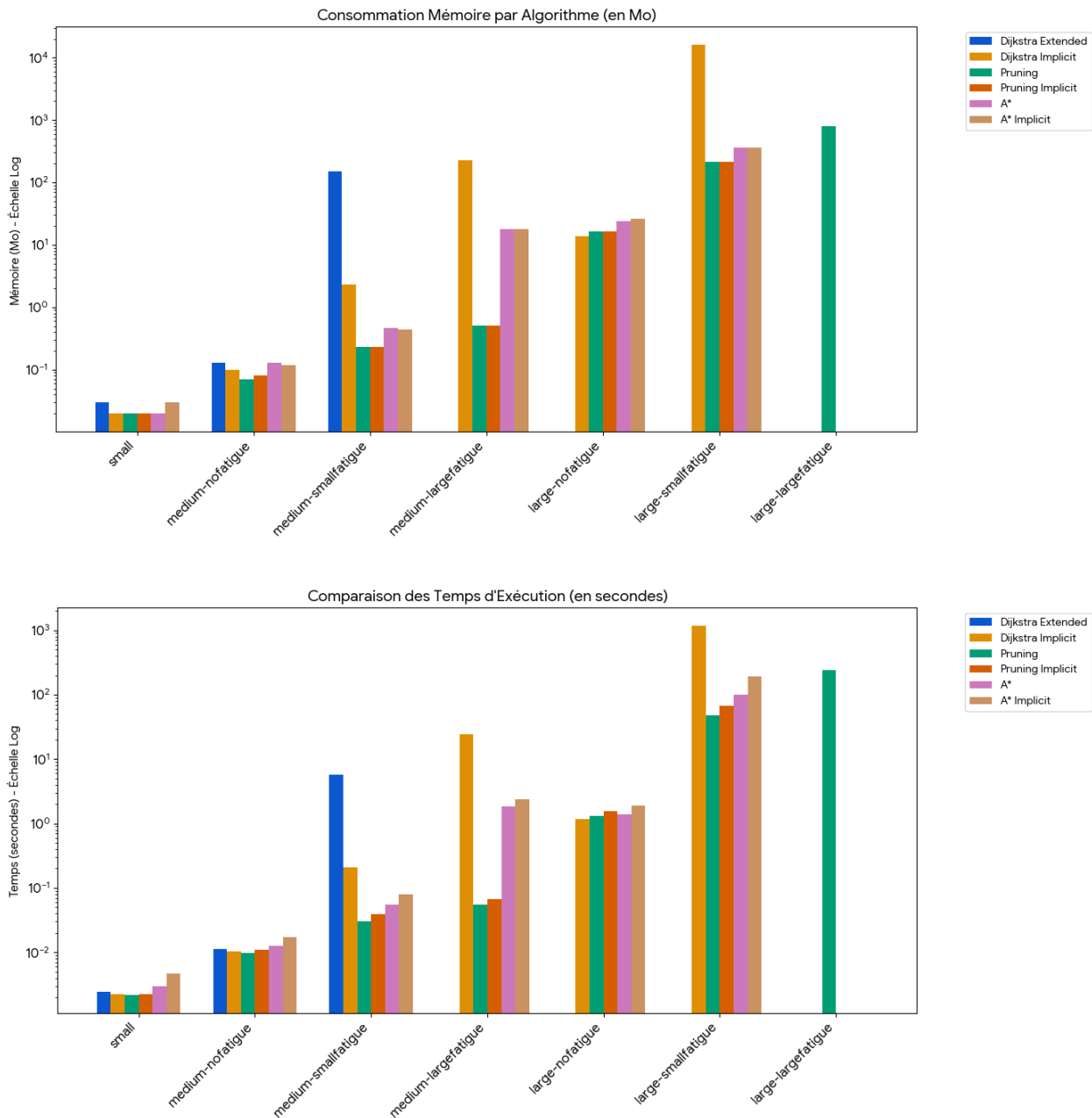
Les fonctions *pruning* et *pruning_implicit* partagent une complexité spatiale théorique de $O(n \times L + m)$, car elles ne stockent que le réseau de base (m) et les listes de compromis par sommet ($n \times L$) où L désigne la taille moyenne des solutions non dominées par sommet (frontière de Pareto). Cependant, il existe une différence : *pruning* crée un dictionnaire d'adjacence local en début de fonction (elle double temporairement le stockage du réseau) alors que *pruning_implicit* utilise des générateurs (*yield*). Pour le temps de calcul, la complexité est de $O(m \log(m) + m \cdot L \cdot (L + \log(n \cdot L))) = O(m \cdot L \cdot \log(n \cdot L))$: l'algorithme se concentre sur $n \cdot L$ états, car chaque sommet n'est plus représentée que par ses L meilleurs compromis "distance/fatigue". On explore chaque chemin m pour ces L variantes, et le facteur $\log(n \cdot L)$ correspond au coût de tri de ces options (opération d'extraction ou d'insertion dans la file de priorité).

c) Contrairement à Dijkstra qui explore le graphe de manière circulaire, *astar* et *astar_implicit* utilisent une fonction d'évaluation $f(n)=g(n)+h(n)$ pour privilégier les chemins pointant vers la destination (au lieu de choisir le sommet non visité de plus petite marque provisoire, on choisira celui qui minimise f , où $g(x)$ = coût pour aller en x). Le succès de cette méthode repose sur la fonction *compute_admissible_heuristic*, qui calcule la distance minimale de chaque sommet vers le but, sans prendre en compte la fatigue. L'heuristique $h(x)$ retenue, définie par le produit de cette distance et de la fatigue actuelle en x , est admissible car elle ne surestime jamais le coût réel restant : la fatigue ne pouvant qu'augmenter au fil du trajet, le coût final sera nécessairement supérieur ou égal à cette estimation.

A^* conserve le même plafond théorique que Dijkstra sur le graphe étendu, soit $O((N_{\text{ext}} + M_{\text{ext}}) \log N_{\text{ext}})$ (où N_{ext} = nombre de sommet du graphe étendu, M_{ext} = nombre d'arcs du graphe étendu) mais est souvent plus rapide grâce à son guidage vers le goal. La version *astar_implicit* optimise mieux l'espace mémoire que *astar*, limitant la consommation à $O(n \cdot F_{\text{max}} + m)$. Cependant, la fonction peut être plus lente que *astar* si haute staleness (recalculs d'arêtes) et est en $O(((N_{\text{ext}} + M_{\text{ext}}) \cdot \text{recalculs}) \log N_{\text{ext}})$. À comparer empiriquement sur les données.

III- Résultats et extensions

a) L'automatisation des tests via le script *benchmark.py* a permis de générer une base de données comparative, à partir de laquelle nous avons construit les graphiques suivants.



L'effondrement du graphe étendu explicite dès l'instance medium-smallfatigue (152,52 Mo) justifie pleinement le passage aux structures implicites. Sur les réseaux de grande taille comme large-smallfatigue, l'algorithme A* (100,7 s) prouve sa supériorité sur le Dijkstra Implicit (1160,4 s) avec une réduction du temps de calcul de plus de 90%, bien que le Pruning reste deux fois plus rapide (47,7 s). Seul *pruning* parvient à conclure le calcul sur large-largefatigue. Il traite ce réseau en 4 minutes (240,5 s) avec une consommation maîtrisée de 800,27 Mo.

b) L'extension du projet aborde un scénario où l'agent ne se contente pas d'un trajet unique, mais doit enchaîner une série de missions .

1. Modélisation et propagation de la fatigue

Contrairement à une recherche classique, l'état initial d'une mission i dépend entièrement de l'état final de la mission $i-1$. Il arrive que la destination d'une mission ne corresponde pas au point de départ de la suivante. Notre algorithme détecte automatiquement ces ruptures et insère des segments de transit. Le problème est traité comme une séquence de segments : $\text{Legs} = \{[\text{Mission}_1], [\text{Transit}_1 \rightarrow 2], [\text{Mission}_2], \dots, [\text{Mission}_k]\}$

Pour garantir que l'encodage des nœuds ($\text{ID} = \text{sommet} \times (\text{F_max} + 1) + \text{fatigue}$) reste valide sur toute la durée du trajet, nous calculons une borne de fatigue globale total_F_max avant la première mission. Cela garantit que le système de coordonnées du graphe ne change pas d'un segment à l'autre, évitant toute corruption de l'espace d'états.

2. Résolution séquentielle (*multi_mission_path*)

Pour chaque trajet (mission ou déplacement inter-mission), on résout le plus court chemin dans le graphe étendu en partant du niveau de fatigue courant. La fatigue finale de chaque trajet devient la fatigue initiale du trajet suivant. C'est une approche gloutonne (chaque trajet est optimisé indépendamment), elle n'est pas nécessairement optimale globalement. Pour chaque segment, nous utilisons un nœud cible virtuel relié à toutes les fatigues possibles de la destination. Cela permet d'identifier en un seul passage le compromis distance/fatigue optimal pour aborder la suite des missions.

L'algorithme effectue k recherches de type Dijkstra sur un graphe étendu comptant N_{ext} nœuds, la complexité s'établit donc à $O(k \cdot (N_{\text{ext}} + M_{\text{ext}}) \log N_{\text{ext}})$

3. Optimisation de l'ordre (*optimal_mission_order*)

L'ordre des missions est une variable critique : puisque la fatigue est cumulative, une mission fatigante effectuée trop tôt pénalise lourdement le reste du trajet : *optimal_mission_order* résout ce problème combinatoire (variante du problème du voyageur de commerce). La fonction énumère exhaustivement toutes les $k!$ permutations des missions via *itertools.permutations*. Pour chaque permutation, elle appelle *multi_mission_path* et conserve la permutation donnant le temps minimal.

En raison de la croissance factorielle de la complexité, cette approche est limitée en pratique à $k \leq 6$ missions : sa complexité vaut $O(k! \times T_{\text{multi}})$, où T_{multi} est le coût d'un appel à *multi_mission_path*

4. Alternative par pruning (*multi_mission_path_pruning*)

Pour gagner en efficacité sur les réseaux denses, on adapte l'algorithme de pruning de pareto aux missions multiples. Chaque segment est résolu par *pruning_single_leg_explicit*, qui ne conserve que les solutions non-dominées.

Pour chaque segment k , on explore m arêtes pour chaque élément de la frontière de Pareto (de taille moyenne L), d'où un $O(k \cdot (m \cdot L \cdot \log(n \cdot L)))$. En pratique, $L \ll F_{\text{total}}$, ce qui rend cette approche plus rapide que Dijkstra sur graphe étendu complet.

c) L'algorithme de Pruning souffre d'une explosion de son espace d'états car il doit explorer une multitude de compromis entre temps et fatigue. Lorsque la mémoire vive devient une ressource critique, l'approche hybride combinant l'Optimisation par colonies de fourmis (ACO) et l'élagage de type *Branch-and-Bound* devient indispensable. Cette stratégie consiste à utiliser une méta-heuristique pour cadrer la recherche exacte. Pour plus de détail sur cette partie, se référer au fichier README_ANTS.

Le rôle précurseur de l'algorithme ACO

L'algorithme implémenté dans ants.py agit comme une unité de reconnaissance rapide. Plutôt que de chercher la solution parfaite immédiatement, les fourmis agissent comme des éclaireurs. Elles parcourent le graphe de manière probabiliste, en laissant derrière elles des pistes de phéromones sur les routes les plus prometteuses. L'objectif de cette phase est de fournir rapidement une borne supérieure de coût, notée T , représentant le meilleur chemin réalisable trouvé après convergence de la colonie. Ce coût T devient le point d'ancrage de toutes les optimisations ultérieures, car il définit le record à battre pour l'algorithme exact.

Élagage par borne de temps

Le premier levier d'optimisation injecté dans le pruning est la borne de temps (*time_limit*). Une fois que les fourmis ont identifié un chemin de coût T , nous avons la certitude que la solution optimale T^* est telle que $T^* \leq T$. En conséquence, lors de l'exploration du graphe par le Pruning, tout chemin partiel dont le coût accumulé dépasse déjà cette valeur T est immédiatement abandonné. Ce mécanisme de coupe-circuit réduit drastiquement le nombre d'insertions dans la file de priorité.

Réduction de l'espace d'états par la borne de fatigue

L'optimisation la plus subtile concerne la hauteur du graphe, c'est-à-dire le niveau de fatigue maximal à explorer. Il existe une limite à la fatigue, que nous nommons *f_bound*, pour n'importe quel chemin dont le coût ne doit pas dépasser une valeur donnée T (voir preuve.pdf). En calculant cette borne avant de lancer le pruning avec *compute_F_bound*, nous réduisons physiquement la taille de l'espace de recherche. Cette réduction physique de la taille du graphe étendu permet d'éviter les erreurs de dépassement de mémoire (MemoryError).

Synthèse de la stratégie Branch-and-Bound

Le flux de travail hybride s'organise donc comme un algorithme de *Branch-and-Bound* : on commence par la phase ACO pour obtenir des bornes, puis on évalue si l'espace de recherche ainsi restreint peut tenir en mémoire grâce au MEMORY_THRESHOLD. Si les conditions sont réunies, le pruning est lancé dans ce périmètre réduit, pour garantir que le chemin final trouvé est le meilleur possible.